
BÁO CÁO PHƯƠNG TRÌNH SCHRÖDINGER

PHỤ THUỘC THỜI GIAN

PHƯƠNG PHÁP DIRECT TIME DISCRETISATION – LEAPFROG

SAI PHÂN KHÔNG GIAN BẬC 2 VÀ BẬC 4

Họ và tên: Nguyễn Trần Gia Bảo

MSSV: 23137003

Môn học: Vật lý tính toán (PHY10532)

Giảng viên: TS. Vũ Quang Tuyên

Ngày 13 tháng 6 năm 2026

Mục lục

1	Giới thiệu và nội dung tính toán	2
2	Cơ sở lý thuyết	2
2.1	Phương trình Schrödinger phụ thuộc thời gian	2
2.2	Bó sóng Gauss ban đầu	2
2.3	Lưới sai phân	3
2.4	Leapfrog với sai phân không gian bậc 2	3
2.5	Leapfrog với sai phân không gian bậc 4	4
2.6	Sai số tán sắc không gian	4
2.7	Điều kiện Courant-Friedrichs-Lewy (CFL)	4
3	Tham số và mô hình rời rạc trong code	5
3.1	Tham số bài toán	5
3.2	Thế năng và điều kiện biên	5
4	Phân tích chương trình Python	6
4.1	Gọi thư viện và khởi tạo bó sóng Gauss	6
4.2	Tạo lưới không gian và thời gian	7
4.3	Kiểm tra điều kiện CFL	7
4.4	Tạo các hệ số Leapfrog bậc 2	7
4.5	Giải TDSE bằng Leapfrog bậc 2	8
4.6	Tạo các hệ số Leapfrog bậc 4	8
4.7	Giải TDSE bằng Leapfrog bậc 4	9
4.8	Ghi hàm sóng ra tập tin văn bản	10
4.9	Tính và ghi chuẩn xác suất	11
4.10	Gán tham số, chạy hàm giải và lưu kết quả	12
4.11	Đọc dữ liệu để vẽ hình	12
4.12	Vẽ và lưu đồ thị bảo toàn xác suất	13
4.13	Tạo các giá trị σ_0 và k_0	13
5	Kết quả và thảo luận	14
5.1	So sánh Leapfrog bậc 2 và bậc 4	14
5.2	Bảo toàn xác suất	15
5.3	Sóng truyền ngược khi $k_0 = -10$	16
5.4	Ảnh hưởng của độ lớn k_0 đến vận tốc và phản xạ	17
5.5	Ảnh hưởng của độ rộng ban đầu σ_0	18
6	Kết luận	20

1. Giới thiệu và nội dung tính toán

Nghiệm số của phương trình Schrödinger phụ thuộc thời gian một chiều được giải bằng phương pháp *Direct Time Discretisation* (DTD) dạng Leapfrog. Đạo hàm theo thời gian được thay bằng sai phân trung tâm, trong khi đạo hàm bậc hai theo không gian được tính theo hai bậc sai phân: sai phân trung tâm bậc 2 và sai phân trung tâm bậc 4.

Chương trình Python sẽ chạy các bước tính toán như sau:

1. Khởi tạo bó sóng Gauss chuẩn hóa tại $x_0 = 5$ với độ rộng σ_0 và số sóng trung tâm k_0 .
2. Xây dựng lưới đều theo x và t , kiểm tra điều kiện Courant-Friedrichs-Lewy (CFL) trước khi giải.
3. Giải TDSE bằng Leapfrog với sai phân không gian bậc 2 và bậc 4.
4. Ghi $\text{Re } \psi$, $\text{Im } \psi$, $|\psi|$ và $|\psi|^2$ ra file data.
5. Tính chuẩn xác suất $N(t) = \int |\psi(x, t)|^2 dx$ bằng quy tắc hình thang.
6. So sánh kết quả bậc 2 và bậc 4 tại $\sigma_0 = 0.5$, $k_0 = 8$.
7. Khảo sát chiều truyền, vận tốc nhóm và phản xạ của bó sóng khi thay đổi k_0 ; chương trình đồng thời quét $\sigma_0 \in \{0.5, 1, 2\}$ và $k_0 \in \{2, 8, 10, -2, -8, -10\}$.

2. Cơ sở lý thuyết

2.1. Phương trình Schrödinger phụ thuộc thời gian

Trong một chiều, phương trình Schrödinger phụ thuộc thời gian có dạng

$$i\hbar \frac{\partial \psi(x, t)}{\partial t} = \hat{H} \psi(x, t) = \left[-\frac{\hbar^2}{2m} \frac{\partial^2}{\partial x^2} + V(x, t) \right] \psi(x, t). \quad (1)$$

Mật độ xác suất được xác định bởi

$$\rho(x, t) = |\psi(x, t)|^2, \quad (2)$$

và điều kiện chuẩn hóa là

$$N(t) = \int_{-\infty}^{+\infty} |\psi(x, t)|^2 dx = 1. \quad (3)$$

Trong mô phỏng hữu hạn, tích phân được lấy trên miền $[x_{\min}, x_{\max}]$.

2.2. Bó sóng Gauss ban đầu

Bó sóng chuẩn hóa được sử dụng trong bài toán này

$$\psi(x, 0) = \frac{1}{(\pi\sigma_0^2)^{1/4}} \exp \left[-\frac{1}{2} \left(\frac{x - x_0}{\sigma_0} \right)^2 \right] \exp(ik_0 x). \quad (4)$$

Hệ số $(\pi\sigma_0^2)^{-1/4}$ làm cho bó sóng được chuẩn hóa trên toàn trục số. Với $x_0 = 5$ và $\sigma_0 = 0.5$, phần đuôi của bó sóng tại hai biên $x = 0$ và $x = 15$ ban đầu rất nhỏ, nên tích phân trên miền mô phỏng xấp xỉ 1.

Nếu V không phụ thuộc x trong miền chuyển động, quan hệ tán sắc liên tục là

$$E(k) = \frac{\hbar^2 k^2}{2m} + V, \quad (5)$$

và vận tốc nhóm của tâm bó sóng là

$$v_g = \frac{1}{\hbar} \frac{dE}{dk} = \frac{\hbar k_0}{m}. \quad (6)$$

Với quy ước $\hbar = m = 1$ trong chương trình, $v_g = k_0$. Vì vậy dấu của k_0 xác định chiều truyền, còn độ lớn $|k_0|$ xác định tốc độ chuyển động của tâm bó sóng trước khi phản xạ ở biên.

2.3. Lưới sai phân

Miền được rời rạc hóa theo

$$x_j = x_{\min} + j\Delta x, \quad t_n = t_{\min} + n\Delta t, \quad \psi_j^n = \psi(x_j, t_n). \quad (7)$$

Đạo hàm thời gian trong công thức Leapfrog được lấy theo sai phân trung tâm

$$\left. \frac{\partial \psi}{\partial t} \right|_j^n = \frac{\psi_j^{n+1} - \psi_j^{n-1}}{2\Delta t} + \mathcal{O}(\Delta t^2). \quad (8)$$

Do công thức sử dụng đồng thời $n-1$, n và $n+1$, bước ψ^1 phải được tính riêng từ ψ^0 bằng Euler tiến.

2.4. Leapfrog với sai phân không gian bậc 2

Đạo hàm không gian bậc hai được xấp xỉ bởi

$$\left. \frac{\partial^2 \psi}{\partial x^2} \right|_j^n = \frac{\psi_{j+1}^n - 2\psi_j^n + \psi_{j-1}^n}{\Delta x^2} + \mathcal{O}(\Delta x^2). \quad (9)$$

Đặt

$$g_j = -\frac{2i\Delta t}{\hbar} \left(\frac{\hbar^2}{m\Delta x^2} + V_j \right), \quad a = \frac{i\hbar\Delta t}{m\Delta x^2}, \quad (10)$$

ta thu được công thức cập nhật

$$\boxed{\psi_j^{n+1} = g_j \psi_j^n + a (\psi_{j+1}^n + \psi_{j-1}^n) + \psi_j^{n-1}.} \quad (11)$$

Bước thời gian đầu được tính bởi

$$\boxed{\psi_j^1 = \psi_j^0 + \frac{1}{2}g_j \psi_j^0 + \frac{1}{2}a (\psi_{j+1}^0 + \psi_{j-1}^0).} \quad (12)$$

2.5. Leapfrog với sai phân không gian bậc 4

Sai phân năm điểm cho đạo hàm bậc hai là

$$\left. \frac{\partial^2 \psi}{\partial x^2} \right|_j^n = \frac{-\psi_{j+2}^n + 16\psi_{j+1}^n - 30\psi_j^n + 16\psi_{j-1}^n - \psi_{j-2}^n}{12\Delta x^2} + \mathcal{O}(\Delta x^4). \quad (13)$$

Đặt

$$g_j = -\frac{2i\Delta t}{\hbar} \left(\frac{5\hbar^2}{4m\Delta x^2} + V_j \right), \quad a = \frac{i\hbar\Delta t}{12m\Delta x^2}, \quad (14)$$

công thức Leapfrog trở thành

$$\boxed{\psi_j^{n+1} = g_j \psi_j^n + 16a (\psi_{j+1}^n + \psi_{j-1}^n) - a (\psi_{j+2}^n + \psi_{j-2}^n) + \psi_j^{n-1}.} \quad (15)$$

Bước đầu tương ứng là

$$\boxed{\psi_j^1 = \psi_j^0 + \frac{1}{2}g_j\psi_j^0 + 8a (\psi_{j+1}^0 + \psi_{j-1}^0) - \frac{1}{2}a (\psi_{j+2}^0 + \psi_{j-2}^0).} \quad (16)$$

2.6. Sai số tán sắc không gian

Khi tác dụng hai điểm sai phân lên mode phẳng e^{ikx} , ta có các khai triển

$$D_{xx}^{(2)} e^{ikx} = \left[-k^2 + \frac{k^4 \Delta x^2}{12} + \mathcal{O}(\Delta x^4) \right] e^{ikx}, \quad (17)$$

$$D_{xx}^{(4)} e^{ikx} = \left[-k^2 + \frac{k^6 \Delta x^4}{90} + \mathcal{O}(\Delta x^6) \right] e^{ikx}. \quad (18)$$

Hai mô phỏng sử dụng cùng sai phân thời gian bậc 2; khác biệt chính trong miền nội đến từ biểu diễn rời rạc của động năng. Với $k_0 = 8$ và $\Delta x = 0.05$, đại lượng $k_0 \Delta x = 0.4$ chưa quá lớn nhưng đủ để sai khác pha tích lũy theo thời gian và trở nên rõ hơn khi bó sóng tiến gần biên.

2.7. Điều kiện Courant-Friedrichs-Lewy (CFL)

Phương pháp tường minh chỉ ổn định (hàm sóng không bị vô hạn) nếu thỏa mãn điều kiện Courant-Friedrichs-Lewy (CFL):

$$\Delta t \leq \frac{1}{2/\Delta x^2 + V_{\max}}, \quad (19)$$

ứng với $\hbar = m = 1$. Với

$$\Delta x = 0.05, \quad \Delta t = \frac{1}{4} \Delta x^2 = 6.25 \times 10^{-4}, \quad V_{\max} = 500,$$

giới hạn trong code là

$$\Delta t_{\max} = \frac{1}{2/(0.05)^2 + 500} = 7.6923 \times 10^{-4}. \quad (20)$$

Do $\Delta t < \Delta t_{\max}$ nên điều kiện CFL được thỏa mãn.

3. Tham số và mô hình rời rạc trong code

3.1. Tham số bài toán

Đại lượng	Giá trị
$x_{\min}, x_{\max}$	0, 15
$t_{\min}, t_{\max}$	0, 1.2
Δx	0.05
Δt	$0.25\Delta x^2 = 0.000625$
N_x	301
N_t	1921
x_0	5
σ_0 trong phép so sánh chính	0.5
k_0 trong phép so sánh chính	8
$\hbar, m$	1, 1
$V_{\max}$	500
Chu kỳ ghi dữ liệu theo thời gian	100 bước, tương ứng $\Delta t_{\text{save}} = 0.0625$

3.2. Thế năng và điều kiện biên

Hàm thế năng được đặt như sau:

```

1  def thenang(x):
2      global V_max
3
4      if 0 <= x <= 15:
5          return 0
6      else:
7          return V_max

```

Hàm trên mô tả thế năng

$$V(x) = \begin{cases} 0, & 0 \leq x \leq 15, \\ V_{\max}, & x < 0 \text{ hoặc } x > 15. \end{cases} \quad (21)$$

Trong chương trình, lưới không gian chỉ được xây dựng trên miền $0 \leq x \leq 15$. Vì vậy, tại mọi điểm lưới được sử dụng trong phép tính, hàm **thenang(x)** đều trả về $V(x) = 0$. Giá trị $V_{\max}$ chỉ được gán cho vùng nằm ngoài miền mô phỏng và không trực tiếp xuất hiện trong phương trình cập nhật hàm sóng.

Hai biên của miền được xem như các giếng thế vuông, tương ứng với điều kiện Dirichlet

$$\psi(0, t) = \psi(15, t) = 0. \quad (22)$$

Điều kiện này được thực hiện bằng cách khởi tạo toàn bộ mảng nghiệm bằng không và chỉ cập nhật các điểm nằm bên trong miền. Đối với phương pháp Leapfrog bậc 2, vòng lặp có dạng

```

1   for j in range(1, N_x-1):
2       ...

```

nên hai điểm biên $j = 0$ và $j = N_x - 1$ không được cập nhật và luôn giữ giá trị $\psi = 0$.

Đối với phương pháp Leapfrog bậc 4, đạo hàm không gian sử dụng khuôn sai phân năm điểm, gồm các nút $j - 2, j - 1, j, j + 1$ và $j + 2$. Vì vậy, chương trình chỉ cập nhật các điểm

```

1   for j in range(2, N_x-2):
2       ...

```

để tránh truy cập ra ngoài mảng. Hai lớp điểm ở mỗi phía, $j = 0, 1$ và $j = N_x - 2, N_x - 1$, được giữ bằng không.

Do cách xử lý này, miền động lực học thực sự của công thức bậc 4 ngắn hơn một khoảng lưới ở mỗi phía so với công thức bậc 2. Vì vậy, sai khác giữa hai kết quả gần thời điểm bó sóng phản xạ không chỉ do bậc chính xác của xấp xỉ đạo hàm không gian, mà còn chịu ảnh hưởng của cách áp đặt điều kiện biên cho khuôn sai phân ba điểm và năm điểm.

4. Phân tích chương trình Python

4.1. Gọi thư viện và khởi tạo bó sóng Gauss

```

1   import numpy as np
2   import matplotlib.pyplot as plt
3
4   def khoitao_psi0(x):
5       global sigma_0, k_0, x_0
6       psi_0 = np.exp(-1/2 * ((x-x_0)/sigma_0)**2) * np.exp(1j*k_0*x) *
           ↪ (np.pi*sigma_0**2)**(-1/4)
7       return psi_0

```

Hai thư viện được nạp ngay đầu chương trình. NumPy cung cấp mảng số phức, hàm mũ, phép lấy modulus, tích phân hình thang và đọc dữ liệu; Matplotlib được dùng ở phần vẽ chuẩn xác suất.

Hàm `khoitao_psi0(x)` nhận toàn bộ lưới không gian dưới dạng một mảng NumPy. Các phép toán trong biểu thức đều được thực hiện theo từng phần tử, nên kết quả `psi_0` là một vector phức có cùng kích thước với `x`. Thừa số

$$\exp\left[-\frac{1}{2}\left(\frac{x-x_0}{\sigma_0}\right)^2\right]$$

tạo đường bao Gauss, `np.exp(1j*k_0*x)` tạo pha sóng phẳng, còn `(np.pi*sigma_0**2)**(-1/4)` là hệ số chuẩn hóa. Ba đại lượng `sigma_0`, `k_0` và `x_0` được lấy từ phạm vi toàn cục; vì vậy mỗi lần vòng quét thay `sigma_0` hoặc `k_0`, hàm tự động tạo bó sóng mới theo bộ tham số hiện hành.

4.2. Tạo lưới không gian và thời gian

```

1 def khoitao_x_t(x_min, x_max, t_min, t_max, dx, dt):
2     N_x = int(round((x_max - x_min) / dx)) + 1
3     N_t = int(round((t_max - t_min) / dt)) + 1
4     x = np.linspace(x_min, x_max, N_x)
5     t = np.linspace(t_min, t_max, N_t)
6     return x, t, N_x, N_t

```

Số nút được tính từ độ dài miền và bước lưới. Phép `round` dùng để làm tròn, còn phép cộng 1 bảo đảm tính cả hai đầu mút. Sau đó `np.linspace` tạo các mảng `x` và `t` chứa chính xác N_x và N_t phần tử. Hàm trả đồng thời bốn đối tượng để các bộ giải có thể dùng trực tiếp lưới và kích thước mảng. Với bộ tham số chính, kết quả là $N_x = 301$ và $N_t = 1921$.

4.3. Kiểm tra điều kiện CFL

```

1 def kiểmtra_CFL(dt, dx, V_max):
2     dieu_kien = 1 / (2 / dx**2 + V_max)
3     if dt <= dieu_kien:
4         print("Dieu kien CFL duoc thoa man.")
5         return True
6     else:
7         print("Khong the hoi tu do vi pham dieu kien CFL.")
8         return False

```

Biến `dieu_kien` chứa giới hạn

$$\Delta t_{\max} = \frac{1}{2/\Delta x^2 + V_{\max}}.$$

Hàm so sánh `dt` với giới hạn này, in thông báo tương ứng và trả về `True` hoặc `False`. Với $\Delta x = 0.05$, $V_{\max} = 500$ và $\Delta t = 0.000625$, nhánh `True` được thực hiện.

4.4. Tạo các hệ số Leapfrog bậc 2

```

1 def khoitao_matran_g_frog_2():
2     global N_x, N_t, x_min, dx, dt, hbar, m
3     g = np.zeros(N_x, dtype=complex)
4     for i in range(N_x):
5         x_i = x_min + i*dx
6         V = thenang(x_i)
7         g[i] = -2j*dt/hbar * (hbar**2/(m*dx**2) + V) #phu thuoc vao x
8     a = 1j*hbar*dt/(m*dx**2)
9     return g, a

```

Mảng `g` có N_x phần tử phức. Vòng lặp xây dựng tọa độ $x_i = x_{\min} + i\Delta x$, gọi `thenang(x_i)` và gán

$$g_i = -\frac{2i\Delta t}{\hbar} \left(\frac{\hbar^2}{m\Delta x^2} + V_i \right).$$

Trong khi g_i có thể thay đổi theo vị trí, a là một hệ số phức dùng chung cho toàn lưới:

$$a = \frac{i\hbar\Delta t}{m\Delta x^2}.$$

Tên hàm có từ *matran*, nhưng code không tạo ma trận hai chiều; cấu trúc được lưu dưới dạng một vector g và một số a , phù hợp với cách cập nhật trực tiếp từng nút.

4.5. Giải TDSE bằng Leapfrog bậc 2

```

1 def giai_leapfrog_bac2():
2     global x_min, x_max, t_min, t_max, dx, dt, x, t, N_x, N_t
3     psi_t_x = np.zeros([N_t, N_x], dtype=np.complex128)
4     psi_0 = khoitao_psi0(x)
5
6     g, a = khoitao_matran_g_frog_2()
7
8     psi_t_x[0] = psi_0
9
10    for j in range(1, N_x-1):
11        psi_t_x[1, j] = psi_t_x[0, j] + 0.5*g[j]*psi_t_x[0, j] +
            ↪ 0.5*a*(psi_t_x[0, j+1]+psi_t_x[0, j-1])
12
13    for n in range(1, N_t-1):
14        for j in range(1, N_x-1):
15            psi_t_x[n+1, j] = g[j]*psi_t_x[n, j] +
            ↪ a*(psi_t_x[n, j+1]+psi_t_x[n, j-1]) + psi_t_x[n-1, j]
16
17    return psi_t_x

```

Mảng `psi_t_x` có dạng $[N_t, N_x]$, nghĩa là chỉ số thứ nhất biểu diễn thời gian và chỉ số thứ hai biểu diễn không gian. Định dạng `np.complex128` được dùng vì hàm sóng có cả phần thực và phần ảo. Hàng đầu tiên `psi_t_x[0]` được gán bằng bố Gauss từ `khoitao_psi0(x)`.

Vòng lặp thứ nhất tính ψ_j^1 cho $j = 1, \dots, N_x - 2$ bằng công thức Euler tiến đã viết dưới dạng các hệ số $g_j/2$ và $a/2$. Hai nút biên không được gán lại nên tiếp tục bằng 0 từ lúc khởi tạo mảng. Vòng lặp kép tiếp theo thực hiện quan hệ ba mức thời gian:

$$\psi_j^{n+1} = g_j\psi_j^n + a(\psi_{j+1}^n + \psi_{j-1}^n) + \psi_j^{n-1}.$$

Sau khi hoàn tất, hàm trả lại toàn bộ lịch sử của hàm sóng thay vì chỉ trả lát thời gian cuối.

4.6. Tạo các hệ số Leapfrog bậc 4

```

1 def khoitao_matran_g_frog_4():
2     global N_x, N_t, x_min, dx, dt, hbar, m
3     g = np.zeros(N_x, dtype=complex)
4     for i in range(N_x):
5         x_i = x_min + i*dx
6         V = thenang(x_i)
7         g[i] = -2*j*dt/hbar * (5*hbar**2/(4*m*dx**2) + V) #phụ thuộc vào x

```

```

8     a = 1j*hbar*dt/(12*m*dx**2)
9     return g, a

```

Cấu trúc hàm giống trường hợp bậc 2, nhưng phần động năng trong g_i và hệ số a được thay theo sai phân năm điểm:

$$g_i = -\frac{2i\Delta t}{\hbar} \left(\frac{5\hbar^2}{4m\Delta x^2} + V_i \right), \quad a = \frac{i\hbar\Delta t}{12m\Delta x^2}.$$

Mảng g vẫn được tính tại mọi nút để cùng chỉ số với mảng hàm sóng.

4.7. Giải TDSE bằng Leapfrog bậc 4

```

1 def giai_leapfrog_bac4():
2     global x_min, x_max, t_min, t_max, dx, dt, x, t, N_x, N_t
3     psi_t_x = np.zeros([N_t, N_x], dtype=np.complex128)
4     psi_0 = khoitao_psi0(x)
5
6     g, a = khoitao_matran_g_frog_4()
7
8     psi_t_x[0] = psi_0
9
10    for j in range(2, N_x-2):
11        psi_t_x[1, j] = psi_t_x[0, j] + 0.5*g[j]*psi_t_x[0, j]
12        ↪ +8*a*(psi_t_x[0, j+1]+psi_t_x[0, j-1]) -
13        ↪ 0.5*a*(psi_t_x[0, j+2]+psi_t_x[0, j-2])
14
15    for n in range(1, N_t-1):
16        for j in range(2, N_x-2):
17            psi_t_x[n+1, j] = g[j]*psi_t_x[n, j] +
18            ↪ 16*a*(psi_t_x[n, j+1]+psi_t_x[n, j-1]) -
19            ↪ a*(psi_t_x[n, j+2]+psi_t_x[n, j-2]) + psi_t_x[n-1, j]
20
21    return psi_t_x

```

Do stencil bậc 4 sử dụng $j \pm 1$ và $j \pm 2$, vòng lặp không gian bắt đầu từ $j=2$ và dừng trước N_x-2 . Hai lớp nút ở mỗi phía vì thế giữ giá trị 0. Ở bước thời gian đầu, các hệ số $8*a$ và $-0.5*a$ chính là một nửa của các hệ số $16*a$ và $-a$ trong công thức Leapfrog đầy đủ. Từ $n = 1$ trở đi, câu lệnh cập nhật tương ứng với

$$\psi_j^{n+1} = g_j \psi_j^n + 16a(\psi_{j+1}^n + \psi_{j-1}^n) - a(\psi_{j+2}^n + \psi_{j-2}^n) + \psi_j^{n-1}.$$

Dầu ra có cùng cấu trúc [thời gian, không gian] như bộ giải bậc 2, nên hai kết quả có thể được xử lý bằng cùng các hàm ghi tập tin.

4.8. Ghi hàm sóng ra tập tin văn bản

```

1 def luu_ketqua_psi(output_name, x, t, y, save_every_time=1, save_every_x =
  ↳ 1):
2     global N_x, N_t, dx, dt, sigma_0, k_0, x_min, x_max, t_min, t_max
3     filename = f"TDSE-ketqua-{output_name}.txt"
4
5     with open(filename, "w", encoding="utf-8") as f:
6         f.write("# KET QUA GIAI PHUONG TRINH SCHRODINGER PHU THUOC THOI
  ↳ GIAN\n\n")
7
8         f.write("#" * 150 + "\n")
9         f.write("# THAM SO DAU VAO\n")
10        f.write("#" * 150 + "\n\n")
11
12        f.write(f"# N_x      = {N_x}\n")
13        f.write(f"# N_t      = {N_t}\n")
14        f.write(f"# x_min    = {x_min}\n")
15        f.write(f"# x_max    = {x_max}\n")
16        f.write(f"# t_min    = {t_min}\n")
17        f.write(f"# t_max    = {t_max}\n")
18        f.write(f"# dx      = {dx}\n")
19        f.write(f"# dt      = {dt}\n")
20        f.write(f"# sigma_0  = {sigma_0}\n")
21        f.write(f"# k_0      = {k_0}\n")
22        f.write(f"# cach_deu_time = {save_every_time}\n")
23
24        f.write("\n" + "#" * 150 + "\n\n")
25
26        f.write(f"#{'t':>15s} {'x':>15s} {'Re_psi':>15s} {'Im_psi':>15s}
  ↳ {'|psi|':>15s} {'|psi|^2':>15s}\n")
27
28        for n in range(0, len(t), save_every_time):
29            for i in range(0, len(x), save_every_x):
30                psi = y[n, i]
31
32                f.write(f" {t[n]:>15.6e} "
33                        f"{x[i]:>15.6e} "
34                        f"{psi.real:>15.6e} "
35                        f"{psi.imag:>15.6e} "
36                        f"{np.abs(psi):>15.6e} "
37                        f"{np.abs(psi)**2:>15.6e}\n")
38
39        f.write("\n")

```

Tên tập tin được ghép từ chuỗi TDSE-ketqua-, tham số output_name và đuôi .txt. Phần đầu tập tin lưu lại kích thước lưới, miền tính, bước lưới và tham số bó sóng; nhờ đó mỗi tập dữ liệu vẫn mang thông tin về phép chạy đã tạo ra nó.

Dòng tiêu đề gồm sáu cột: t , x , $\text{Re}\psi$, $\text{Im}\psi$, $|\psi|$ và $|\psi|^2$. Vòng ngoài duyệt thời gian với bước save_every_time, còn vòng trong duyệt không gian với bước save_every_x. Biến psi=y[n,i] lấy một phần tử phức rồi tách các đại lượng cần ghi. Định dạng 15.6e lưu số

theo ký hiệu khoa học với sáu chữ số sau dấu thập phân. Dòng trống sau mỗi bước thời gian phân tách các khối dữ liệu, thuận tiện cho lệnh vẽ mặt bằng gnuplot.

4.9. Tính và ghi chuẩn xác suất

```

1 def luu_file_xac_xuat(output_name, x, t, y, save_every_time=1):
2     global N_x, N_t, dx, dt, sigma_0, k_0, x_min, x_max, t_min, t_max
3     filename = f"TDSE-ketqua-{output_name}-xacxuat.txt"
4
5     with open(filename, "w", encoding="utf-8") as f:
6         f.write("# KET QUA GIAI PHUONG TRINH SCHRODINGER PHU THUOC THOI
7             ↪ GIAN\n\n")
8
9         f.write("#" * 150 + "\n")
10        f.write("# THAM SO DAU VAO\n")
11        f.write("#" * 150 + "\n\n")
12
13        f.write(f"# N_x      = {N_x}\n")
14        f.write(f"# N_t      = {N_t}\n")
15        f.write(f"# x_min    = {x_min}\n")
16        f.write(f"# x_max    = {x_max}\n")
17        f.write(f"# t_min    = {t_min}\n")
18        f.write(f"# t_max    = {t_max}\n")
19        f.write(f"# dx      = {dx}\n")
20        f.write(f"# dt      = {dt}\n")
21        f.write(f"# sigma_0  = {sigma_0}\n")
22        f.write(f"# k_0      = {k_0}\n")
23        f.write(f"# cach_deu_time = {save_every_time}\n")
24
25        f.write("\n" + "#" * 150 + "\n\n")
26
27        f.write(f"#{'t':>15s} {'Integral_|psi|^2_dx':>25s}\n")
28
29        for n in range(0, len(t), save_every_time):
30
31            prob_x = np.abs(y[n, :])**2
32            tong_xac_xuat = np.trapz(prob_x, x)
33
34            f.write(f" {'t[n]:>15.6e} "
35                f" {'tong_xac_xuat:>25.6e}\n")

```

Hàm thứ hai tạo tập tin có hậu tố `-xacxuat.txt`. Tại mỗi thời điểm được chọn, câu lệnh `np.abs(y[n, :])**2` tạo mật độ xác suất trên toàn bộ lưới. `np.trapz(prob_x, x)` thực hiện tích phân hình thang

$$N(t_n) \approx \int_{x_{\min}}^{x_{\max}} |\psi(x, t_n)|^2 dx.$$

4.10. Gán tham số, chạy hàm giải và lưu kết quả

```

1 sigma_0 = 0.5
2 dx = 0.05
3 dt = 1/4*dx**2
4 k_0 = 8
5 x_0 = 5
6
7 hbar = 1
8 m = 1
9
10 t_min = 0
11 t_max = 1.2
12
13 x_min = 0
14 x_max = 15
15
16 V_max = 500
17 kiểmtra_CFL(dt, dx, V_max)
18
19 x, t, N_x, N_t = khoitao_x_t(x_min, x_max, t_min, t_max, dx, dt)
20 psi_t_x_bac_2 = giai_leapfrog_bac2()
21 psi_t_x_bac_4 = giai_leapfrog_bac4()
22 lưu_ketqua_psi("Leapfrog_Bac2", x, t, y=psi_t_x_bac_2, save_every_time=100)
23 lưu_file_xac_xuat("Leapfrog_Bac2", x, t, y=psi_t_x_bac_2,
    ↪ save_every_time=100)
24 lưu_ketqua_psi("Leapfrog_Bac4", x, t, y=psi_t_x_bac_4, save_every_time=100)
25 lưu_file_xac_xuat("Leapfrog_Bac4", x, t, y=psi_t_x_bac_4,
    ↪ save_every_time=100)

```

Khối lệnh này xác lập cấu hình mô phỏng chính: $\sigma_0 = 0.5$, $k_0 = 8$, $x_0 = 5$, $\Delta x = 0.05$, $\Delta t = \Delta x^2/4$, $\hbar = m = 1$, miền $x \in [0, 15]$, miền $t \in [0, 1.2]$ và $V_{\max} = 500$.

Sau lệnh kiểm tra CFL, lưới được tạo một lần rồi dùng chung cho cả hai bộ giải. Hai biến `psi_t_x_bac_2` và `psi_t_x_bac_4` chứa toàn bộ nghiệm số.

4.11. Đọc dữ liệu để vẽ hình

```

1 import matplotlib.pyplot as plt
2 plt.style.use('sci.mplstyle')
3 t_xacsuat_bac2, N_bac2 = np.loadtxt("TDSE-ketqua-Leapfrog_Bac2-xacxuat.txt",
    ↪ unpack = True, comments = "#")
4 t_xacsuat_bac4, N_bac4 = np.loadtxt("TDSE-ketqua-Leapfrog_Bac4-xacxuat.txt",
    ↪ unpack = True, comments = "#")
5
6 x_bac2, t_bac2, psi_bac2 = np.loadtxt("TDSE-ketqua-Leapfrog_Bac2.txt", unpack
    ↪ = True, comments = "#", usecols=(0,1,4))
7 x_bac4, t_bac4, psi_bac4 = np.loadtxt("TDSE-ketqua-Leapfrog_Bac4.txt", unpack
    ↪ = True, comments = "#", usecols=(0,1,4))
8 plt.figure(figsize=(7, 6))
9

```

```
10 fig, ax = plt.subplots(1, 2, figsize=(12, 5), sharey=True)
```

4.12. Vẽ và lưu đồ thị bảo toàn xác suất

```
1 ax[0].plot(t_xacsuat_bac2, N_bac2, linewidth = 3)
2 ax[0].set_xlabel(r"$t$")
3 ax[0].set_ylabel(r"$N(t)$")
4 ax[0].set_title("Leapfrog bac 2")
5 ax[0].grid(True, alpha=0.3)
6
7 ax[1].plot(t_xacsuat_bac4, N_bac4, linewidth = 3)
8 ax[1].set_xlabel(r"$t$")
9 ax[1].set_title("Leapfrog bac 4")
10 ax[1].grid(True, alpha=0.3)
11
12 ax[0].set_ylim(0,2)
13
14 ax[0].set_xlim(0,1)
15 ax[1].set_xlim(0,1)
16
17 plt.tight_layout()
18 plt.savefig("baotoan_xacsuat.pdf", bbox_inches="tight")
19 plt.show()
```

4.13. Tạo các giá trị σ_0 và k_0

```
1 sigma_0_list = [0.5, 1, 2]
2 k_0_list = [2, 8, 10, -2, -8, -10]
3
4 for sigma_value in sigma_0_list:
5     for k_value in k_0_list:
6         sigma_0 = sigma_value
7         k_0 = k_value
8         filename = (f"Leapfrog_Bac4_sigma={sigma_0}_k0={k_0}")
9
10        psi_t_x_bac_4 = giai_leapfrog_bac4()
11
12        luu_ketqua_psi(
13            filename,
14            x,
15            t,
16            y=psi_t_x_bac_4,
17            save_every_time=100
18        )
19
20        luu_file_xac_xuat(
21            filename,
22            x,
23            t,
```

```

24         y=psi_t_x_bac_4,
25         save_every_time=100
26     )

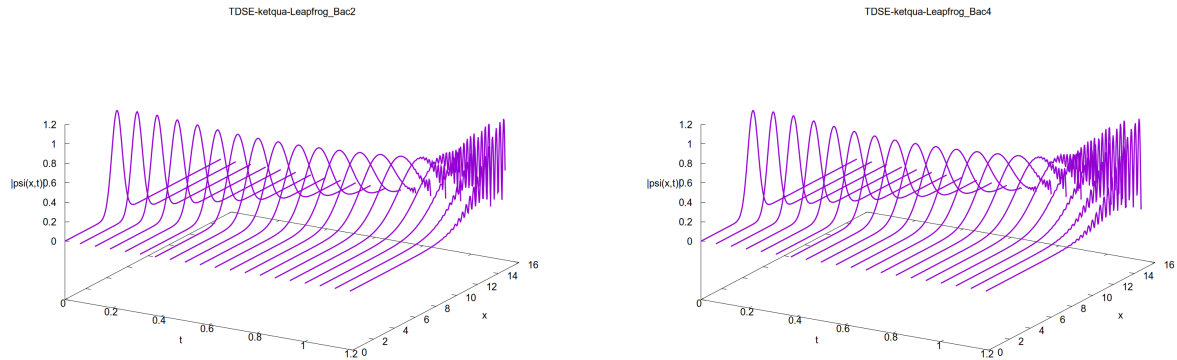
```

Hai danh sách tạo $3 \times 6 = 18$ tổ hợp tham số. Ở mỗi vòng lặp, các biến toàn cục `sigma_0` và `k_0` được cập nhật trước khi gọi `giai_leapfrog_bac4()`, nên hàm `khoitao_psi0` bên trong bộ giải tạo đúng bó sóng mới. Lưới, bước thời gian, thế năng và các hằng số vật lý vẫn giữ nguyên.

5. Kết quả và thảo luận

5.1. So sánh Leapfrog bậc 2 và bậc 4

Hai hình dưới đây được tạo với cùng bộ tham số $\sigma_0 = 0.5$, $k_0 = 8$, $x_0 = 5$, $\Delta x = 0.05$ và $\Delta t = 0.000625$.



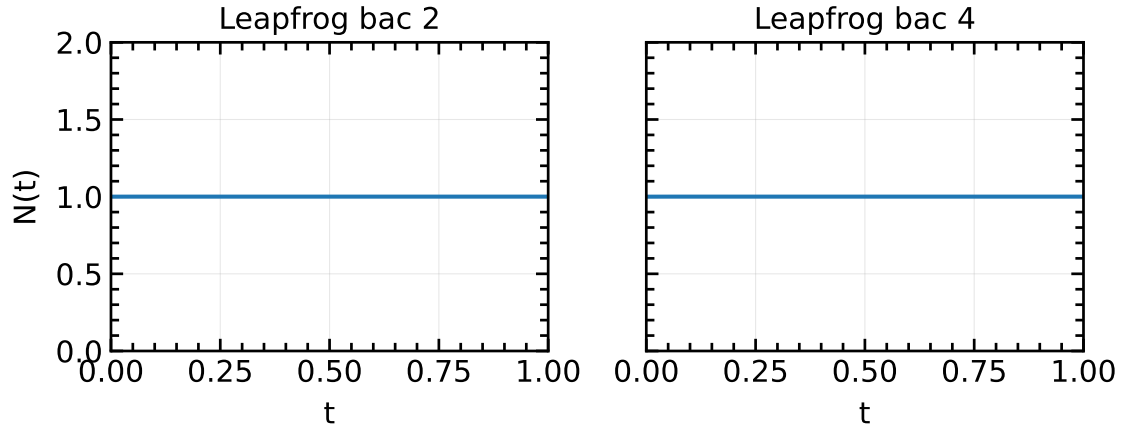
(a) Leapfrog với sai phân không gian bậc 2.

(b) Leapfrog với sai phân không gian bậc 4.

Hình 1: Sự tiến hóa của $|\psi(x, t)|$ khi $\sigma_0 = 0.5$ và $k_0 = 8$.

Ở cả hai kết quả, bó sóng xuất phát từ $x_0 = 5$, truyền theo chiều $+x$ với vận tốc nhóm $v_g = 8$ và giãn rộng theo thời gian. Phần đuôi bó sóng chạm tường $x = 15$ trước tâm, sau đó phản xạ ngược lại và giao thoa với sóng tới, tạo thành các vân gần biên. Sai khác giữa Hình 1a và Hình 1b chủ yếu do sai số pha rời rạc và cách xử lý nút biên khác nhau của phương pháp bậc 2 và bậc 4.

5.2. Bảo toàn xác suất



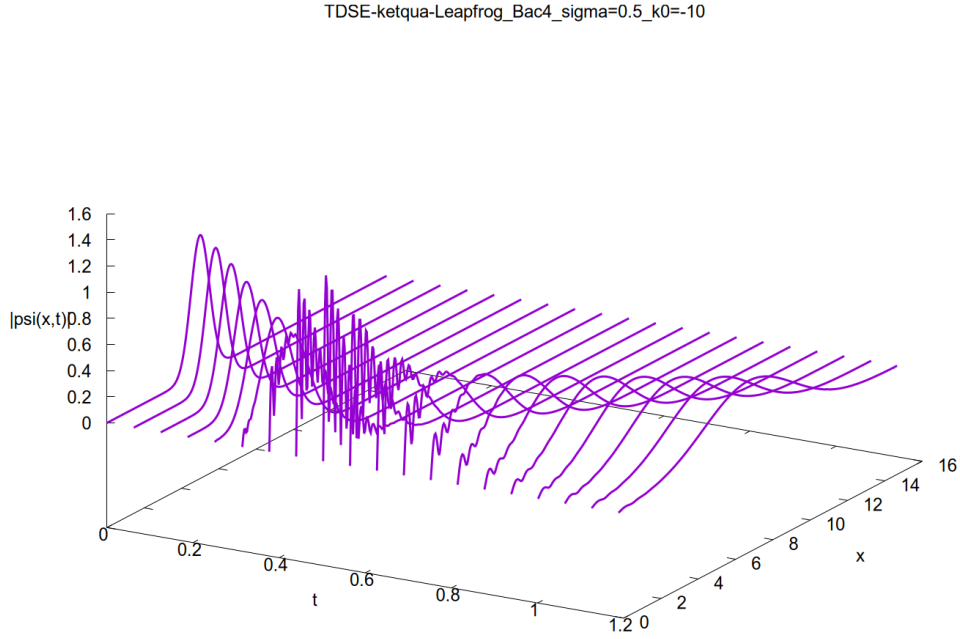
Hình 2: Chuẩn xác suất $N(t)$ của Leapfrog bậc 2 và bậc 4. Hai ô dùng chung giới hạn trục tung.

Tại $t = 0$, hệ số chuẩn hóa của bó Gauss cho $N(0) \simeq 1$. Các giá trị thu được xấp xỉ:

t	$N_{\text{bậc } 2}(t)$	$N_{\text{bậc } 4}(t)$
0	1.000000	1.000000
0.0625	1.059575	1.059116
0.5000	1.062325	1.062457
1.0000	1.062325	1.062448

Sau giai đoạn đầu, $N(t)$ gần như không đổi đối với cả hai phương pháp. Điều này cho thấy xác suất được bảo toàn theo thời gian.

5.3. Sóng truyền ngược khi $k_0 = -10$



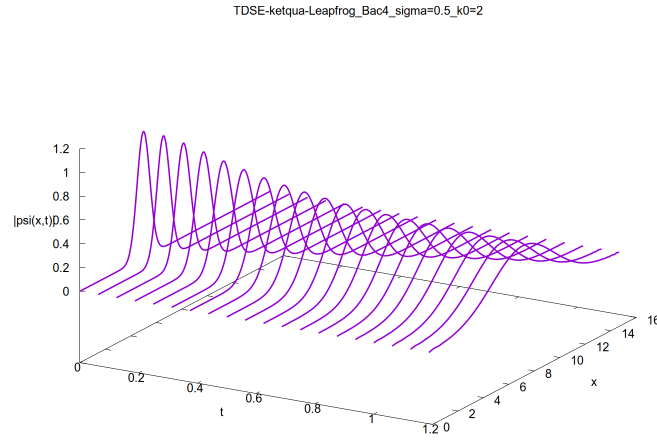
Hình 3: Leapfrog bậc 4 với $\sigma_0 = 0.5$ và $k_0 = -10$: bó sóng truyền về phía $-x$, phản xạ ở biên trái rồi đổi chiều.

Với $k_0 = -10$, vận tốc nhóm ban đầu là $v_g = -10$. Tâm bó sóng cách biên trái một đoạn 5, nên thời điểm va chạm ước lượng là

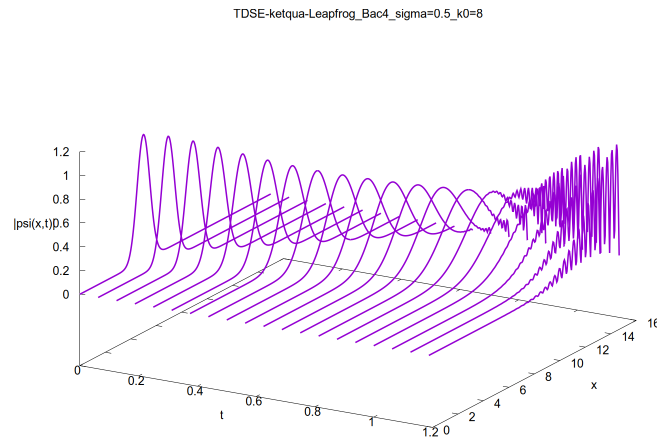
$$t \approx \frac{5}{10} = 0.5. \quad (23)$$

Trước thời điểm này, đỉnh xác suất dịch về trái. Khi thành phần tới chồng lên thành phần phản xạ, $|\psi|$ xuất hiện hiện tượng giao thoa. Sau khi phản xạ hoàn tất, bó sóng chuyển động về phía $+x$.

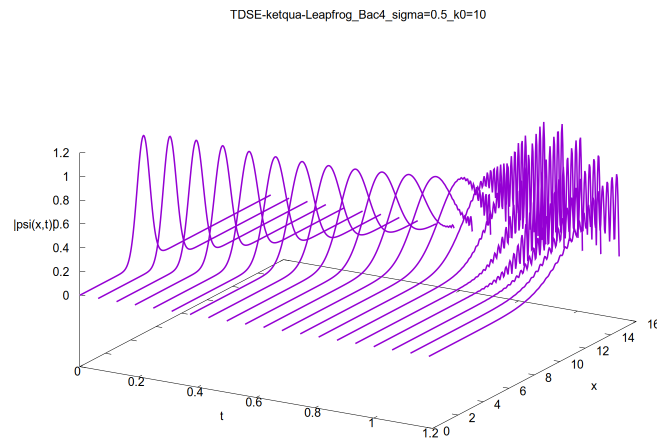
5.4. Ảnh hưởng của độ lớn k_0 đến vận tốc và phản xạ



(a) $k_0 = 2$.



(b) $k_0 = 8$.



(c) $k_0 = 10$.

Hình 4: Sóng truyền về phía $+x$ với cùng $\sigma_0 = 0.5$ và ba giá trị số sóng trung tâm.

Các thời điểm để tâm bó sóng đi từ $x_0 = 5$ đến biên phải $x = 15$ theo vận tốc nhóm liên tục là

$$t \approx \frac{15 - 5}{k_0}. \quad (24)$$

Suy ra:

k_0	v_g	t ước lượng
2	2	5.0
8	8	1.25
10	10	1.0

Với $k_0 = 2$, thời gian mô phỏng 1.2 ngắn hơn nhiều so với thời gian chạm biên. Hình 4a vì vậy chủ yếu thể hiện chuyển động tịnh tiến và sự giãn rộng của bó sóng, chưa hình thành phản xạ rõ rệt.

Với $k_0 = 8$, tâm bó sóng chưa hoàn toàn đạt biên phải tại $t = 1.2$, nhưng phần đuôi đã tương tác với vùng tường. Hình 4b cho phép quan sát giai đoạn bắt đầu chồng chập giữa sóng tới và sóng phản xạ.

Với $k_0 = 10$, tâm bó sóng đạt biên vào khoảng $t = 1$. Khoảng thời gian còn lại đủ để thành phần phản xạ quay trở lại miền trong. Trong vùng hai thành phần cùng tồn tại, mật độ xác suất có các cực đại và cực tiểu liên tiếp. Đây là các vân giao thoa do tổng biên độ

$$\psi(x, t) = \psi_{\text{tới}}(x, t) + \psi_{\text{phản xạ}}(x, t) \quad (25)$$

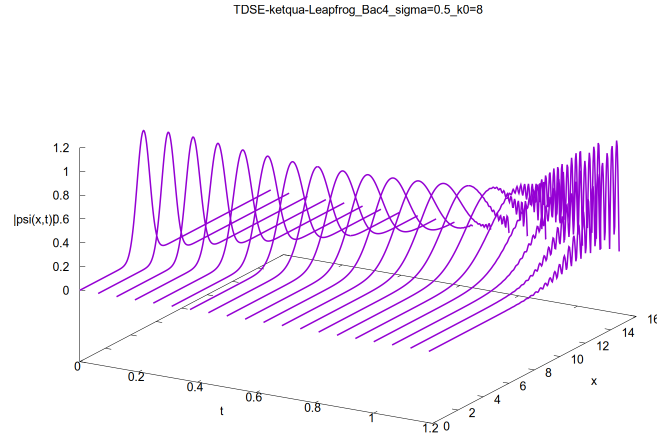
thỏa

$$|\psi|^2 = |\psi_{\text{tới}}|^2 + |\psi_{\text{phản xạ}}|^2 + 2 \operatorname{Re}(\psi_{\text{tới}}^* \psi_{\text{phản xạ}}). \quad (26)$$

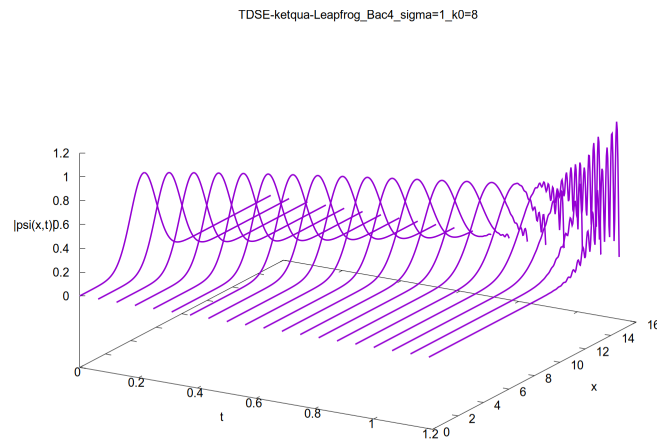
Số vân tăng khi $|k_0|$ lớn vì bước sóng de Broglie $\lambda = 2\pi/|k_0|$ giảm.

5.5. Ảnh hưởng của độ rộng ban đầu σ_0

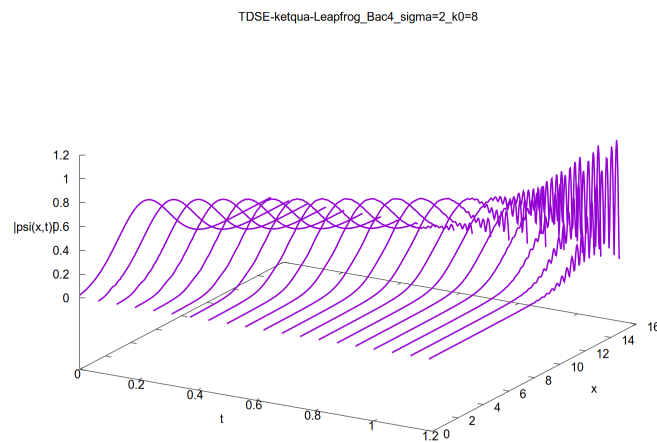
Để khảo sát riêng ảnh hưởng của độ rộng bó sóng, số sóng trung tâm được giữ cố định tại $k_0 = 8$, trong khi σ_0 lần lượt nhận các giá trị 0.5, 1 và 2. Kết quả mô phỏng bằng phương pháp Leapfrog bậc 4 được trình bày trong Hình 5.



(a) $\sigma_0 = 0.5$.



(b) $\sigma_0 = 1$.



(c) $\sigma_0 = 2$.

Hình 5: Sự tiến hóa của bó sóng khi giữ cố định $k_0 = 8$ và thay đổi độ rộng ban đầu σ_0 .

Theo Phương trình (4), σ_0 xác định độ rộng ban đầu của bó sóng. Khi σ_0 tăng, bó sóng

rộng hơn trong không gian nhưng phân bố số sóng hẹp hơn, nên mức độ giãn theo thời gian giảm.

Với $\sigma_0 = 0.5$ trong Hình 5a, bó sóng hẹp và giãn rõ nhất. Khi $\sigma_0 = 1$ trong Hình 5b, bó sóng rộng hơn và giãn ít hơn. Với $\sigma_0 = 2$ trong Hình 5c, bó sóng rộng ngay từ đầu và giãn ít nhất, nhưng phần đuôi có thể chạm biên sớm hơn.

Do $k_0 = 8$ được giữ cố định trong cả ba trường hợp, vận tốc nhóm và quỹ đạo của tâm bó sóng gần như không đổi. Sau khi gặp biên, sóng phản xạ chồng chập với sóng tới và tạo thành các vân giao thoa.

6. Kết luận

Trong bài này, phương pháp DTD–Leapfrog được sử dụng với hai dạng sai phân không gian bậc 2 và bậc 4. Bước đầu tiên được tính bằng Euler tiến, sau đó nghiệm được cập nhật bằng công thức ba mức thời gian.

Với $\sigma_0 = 0.5$ và $k_0 = 8$, cả hai phương pháp đều cho thấy bó sóng truyền sang phải, giãn rộng và phản xạ tại biên. Chuẩn xác suất tăng từ 1 lên khoảng 1.062 ở bước đầu, sau đó gần như được bảo toàn.

Các kết quả với $k_0 = -10, 2, 8, 10$ cho thấy dấu của k_0 quyết định hướng truyền, còn $|k_0|$ càng lớn thì bó sóng chạm biên và xuất hiện giao thoa càng sớm.